ML Library Primer

NeonTreeEvaluation Working Group

Keep this open while reading the code

Contents

1 The Big Picture **1**

2 NumPy: Images Are Grids of Numbers **1**
2.1 Why a separate library at all? . . . 1
2.2 The array: NumPy's one big idea . . . 1
2.3 How images become arrays . . . 2
2.4 Slicing: picking out parts of an array . . . 2
2.5 Vectorised operations: no for loops over pixels . . . 3

3 PyTorch: NumPy That Can Learn **3**
3.1 Why not just use NumPy? . . . 3
3.2 Tensors . . . 4
3.3 Moving data to the GPU . . . 4
3.4 Automatic differentiation . . . 4
3.5 The training loop . . . 5
3.6 Datasets and DataLoaders . . . 5

4 Torchvision: The Image-Specific Toolkit **6**
4.1 Transforms . . . 6
4.2 Pre-trained models and transfer learning . . . 7
4.3 Inference: running the model on new data . . . 8

5 The Full Pipeline in One View **8**

The Big Picture

Our task is to automatically detect individual tree crowns in aerial imagery collected by the National Ecological Observatory Network (NEON). The dataset includes:

- **RGB imagery** – standard colour photos taken from the air (think Google Maps, but scientific quality).
- **LiDAR-derived canopy height** – a laser rangefinder sweep that gives us a height value for every point on the ground, letting us distinguish tall trees from shrubs and grass even when everything looks green in the photo.
- **Hyperspectral imagery** – hundreds of light wavelengths beyond what the human eye can see, which can distinguish tree species by how their leaves reflect light.
- **Labels** – for each plot, expert annotators drew bounding boxes around every visible tree crown. These are our training targets.

A machine learning model will look at a patch of this imagery and output a list of boxes (and possibly pixel-level masks) indicating where it thinks tree crowns are. To build, train, and run such a model in Python we need three libraries: **NumPy**, **PyTorch**, and **Torchvision**. This document explains what each one does and why we reach for it.

Note

These libraries do not know anything about trees. They are general-purpose mathematical and engineering tools that happen to be the lingua franca of computer vision research. You do not need to understand the mathematics deeply to work with them – but you do need to understand their data representations, because every bug you will encounter ultimately comes down to a misunderstanding about how data is shaped or stored.

NumPy: Images Are Grids of Numbers

Why a separate library at all?

Python lists can store numbers, so why not just use those? The problem is speed. A single RGB image tile from the NEON dataset might be 400×400 pixels, meaning 480 000 numbers. A Python list of 480 000 numbers is slow because Python checks the type of each number individually every time you do anything with it. NumPy avoids this by storing all the numbers in a contiguous block of memory and running operations on the whole block at once in compiled C code. The difference is typically 100–1000x faster.

The array: NumPy's one big idea

NumPy has essentially one data structure: the *array* (called `ndarray` internally). Everything else is operations on arrays. An array is a grid of numbers, all of the same type, with a fixed shape.

```
import numpy as np

# A 1-D array -- just a list of numbers
heights = np.array([15.2, 8.7, 22.1, 5.0])

# A 2-D array -- a grid (like a spreadsheet)
patch = np.zeros((400, 400))  # 400 rows, 400 columns, all zeros

# A 3-D array -- what a single RGB image looks like
rgb_image = np.zeros((400, 400, 3))  # height x width x channels
```

The shape of an array is the most important thing to keep in your head at all times. `rgb_image.shape` returns `(400, 400, 3)`, which means 400 rows, 400 columns, and 3 values per pixel (red, green, blue, each ranging from 0 to 255).

How images become arrays

Every image you have ever seen on a screen is a grid of pixels. Each pixel is just a few numbers. An RGB pixel is three numbers: how much red, how much green, how much blue. A single NEON RGB tile therefore becomes a `(height, width, 3)` array of integers.

The LiDAR canopy height raster is a `(height, width, 1)` array – one number per pixel, representing height above ground in metres. You will sometimes see this stored as `(height, width)` (no trailing dimension) – these two shapes refer to the same physical data; the only difference is notational.

The hyperspectral data has 369 wavelength bands, making each tile a `(height, width, 369)` array. That is a lot of channels, which is part of why ML approaches are appealing – no human can manually reason about 369 simultaneous measurements per pixel.

Intuition

Think of a 3-D array as a stack of 2-D grids, one per channel. For an RGB image: layer 0 is the "red intensity" map, layer 1 is "green intensity", layer 2 is "blue intensity". For hyperspectral data: layer 0 is how brightly the scene reflects 380 nm ultraviolet light, layer 1 is 382 nm, and so on up to near-infrared.

Slicing: picking out parts of an array

You will frequently need to crop a region out of an image, or isolate one channel. NumPy uses the same slice syntax as Python lists, extended to multiple dimensions:

```
# Load a NEON tile (rasterio is the library that reads GeoTIFF files;
# it hands the data back to you as a numpy array)
import rasterio
with rasterio.open("data/RGB/SJER_006.tif") as src:
    tile = src.read()  # shape: (3, height, width) -- note: channels
        first!

# Crop to a 256x256 patch starting at pixel (100, 200)
patch = tile[:, 100:356, 200:456]
# tile[:, ...]  means "all channels"
# 100:356       means rows 100 up to (but not including) 356
# 200:456       means columns 200 up to (but not including) 456

# Pull out just the red channel
red_channel = tile[0, :, :]    # shape: (height, width)

# Pull out just the green channel
green_channel = tile[1, :, :] # shape: (height, width)
```

Note

Notice that `rasterio` returns images as `(channels, height, width)` – channels first – while the convention you will see elsewhere (e.g. when building arrays by hand) is often `(height, width, channels)` – channels last. This inconsistency is a common source of bugs. Always check `array.shape` when loading data from a new source.

Vectorised operations: no for loops over pixels

The most important habit to develop with NumPy is to avoid writing Python `for` loops over array elements. If you find yourself writing `for i in range(height): for j in range(width): ...`, there is almost certainly a NumPy function that does it in a single line, thousands of times faster.

```
# Bad: slow Python loop
for i in range(400):
    for j in range(400):
        tile[0, i, j] = tile[0, i, j] / 255.0

# Good: vectorised -- the division applies to every element at once
tile = tile / 255.0

# Normalise each channel to zero mean and unit variance
mean = tile.mean(axis=(1, 2), keepdims=True)  # shape: (3, 1, 1)
std  = tile.std(axis=(1, 2), keepdims=True)
tile_normalised = (tile - mean) / std
```

The `axis` parameter tells NumPy which dimensions to collapse. In the example above, we want one mean per channel, so we average over axes 1 and 2 (height and width) while keeping axis 0 (channels) intact.

PyTorch: NumPy That Can Learn

Why not just use NumPy?

NumPy is excellent for manipulating data, but it cannot train a neural network on its own. There are two things it is missing:

1. **GPU support.** Training a modern object detector on a dataset the size of NeonTreeEvaluation on a CPU would take weeks. A GPU performs the same arithmetic but has thousands of cores running in parallel, reducing this to hours. NumPy has no GPU support.
2. **Automatic differentiation.** Training a neural network means repeatedly computing "how should each parameter change to reduce the error?" This requires calculating gradients – derivatives of the error with respect to every parameter. Doing this by hand for a model with millions of parameters is impossible. PyTorch does it automatically.

PyTorch solves both problems. Its fundamental data structure is the *tensor*, which is essentially a NumPy array that can live on a GPU and can track gradients.

Tensors

A tensor behaves almost identically to a NumPy array:

```
import torch
import numpy as np

# Create a tensor directly
t = torch.zeros((3, 400, 400))   # same syntax as np.zeros

# Convert from numpy (shares memory -- no data is copied)
arr = np.random.rand(3, 400, 400).astype(np.float32)
t = torch.from_numpy(arr)

# Convert back to numpy
arr_back = t.numpy()

# Arithmetic works the same way
t_normalised = (t - t.mean()) / t.std()
print(t_normalised.shape)   # torch.Size([3, 400, 400])
```

The main practical difference you will notice is the method names. NumPy uses `np.zeros`, PyTorch uses `torch.zeros`. NumPy uses `arr.shape`, PyTorch also uses `t.shape` but wraps the result in a `torch.Size` object (which behaves like a tuple).

Moving data to the GPU

When a GPU is available, you move tensors to it explicitly:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)  # "cuda" on a GPU machine, "cpu" otherwise
```

```
t = t.to(device)  # move tensor to GPU (or keep on CPU if no GPU)
```

Once on the GPU, all operations on that tensor happen on the GPU automatically. The catch is that you cannot mix GPU and CPU tensors in the same operation – both operands must be on the same device. This is probably the most common runtime error you will encounter: `RuntimeError: Expected all tensors to be on the same device.`

Automatic differentiation

This is the feature that makes PyTorch indispensable for training. Every tensor can optionally record a history of the operations applied to it so that PyTorch can later compute gradients automatically. You will not need to implement this yourself – the training loop does it for you – but understanding what is happening underneath prevents confusion.

```
# Model parameters need gradient tracking
weight = torch.tensor([0.5], requires_grad=True)

# A trivial "model": prediction = weight * input
x = torch.tensor([2.0])
prediction = weight * x   # = 1.0

# Suppose the true answer is 3.0; compute squared error
loss = (prediction - 3.0) ** 2   # = 4.0

# Ask PyTorch to compute d(loss)/d(weight)
loss.backward()
print(weight.grad)   # tensor([-8.]) -- loss decreases if we increase
    weight
```

`loss.backward()` is the single call that triggers PyTorch to differentiate the entire computation graph. In a real model with millions of parameters, the same call computes all the gradients simultaneously.

The training loop

In practice you will see the same four-step pattern repeated for every batch of training data:

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for images, targets in dataloader:
    images = images.to(device)
    targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

    # 1. Forward pass: run the model, get predictions and loss
    loss_dict = model(images, targets)
    loss = sum(loss_dict.values())

    # 2. Zero gradients from the previous batch (must do this every
        iteration)
    optimizer.zero_grad()

    # 3. Backward pass: compute gradients
    loss.backward()

    # 4. Update parameters in the direction that reduces loss
    optimizer.step()
```

The optimizer (`torch.optim.Adam` here) implements the rule for updating parameters given their gradients. Adam is a standard choice that works well across a wide range of problems without much tuning.

Datasets and DataLoaders

PyTorch has a pair of abstractions for feeding data to a model in a structured way.

A **Dataset** knows how to return one example (one image and its annotations) given an index. You implement one by subclassing `torch.utils.data.Dataset` and defining two methods:

```
class NeonDataset(torch.utils.data.Dataset):
    def __init__(self, image_paths, annotation_paths, transform=None):
        self.image_paths = image_paths
        self.annotation_paths = annotation_paths
        self.transform = transform

    def __len__(self):
        # How many examples are in this dataset?
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Load and return the idx-th example
        image = load_image(self.image_paths[idx])     # returns a tensor
        boxes = load_boxes(self.annotation_paths[idx]) # returns a
            tensor of [x1,y1,x2,y2]

        if self.transform:
            image = self.transform(image)

        target = {"boxes": boxes, "labels": torch.ones(len(boxes),
            dtype=torch.long)}
        return image, target
```

A **DataLoader** wraps a Dataset and handles batching, shuffling, and parallel loading:

```
dataset = NeonDataset(image_paths, annotation_paths,
    transform=preprocess)
dataloader = torch.utils.data.DataLoader(
    dataset,
    batch_size=4,          # process 4 images at once
    shuffle=True,          # randomise order each epoch
    num_workers=4,         # load data in parallel on 4 CPU workers
)

# The training loop iterates over the DataLoader
for images, targets in dataloader:
    ...  # images is now a batch of 4 tensors stacked together
```

Note

When the DataLoader collects individual examples into a batch, the shape of each image tensor changes from `(C, H, W)` to `(B, C, H, W)`, where `B` is the batch size. This leading batch dimension is ubiquitous in PyTorch code; if you see a tensor with an unexpected extra leading dimension, it is usually the batch.

Torchvision: The Image-Specific Toolkit

Torchvision is an official PyTorch companion library that provides three things: a collection of image transforms, pre-trained model architectures, and utilities for common vision datasets. We care primarily about the first two.

Transforms

Raw imagery cannot be fed directly into most models. The images need to be pre-processed to a consistent size, normalised so that pixel values have similar magnitudes, and sometimes augmented (randomly flipped, cropped, etc.) to help the model generalise. Torchvision provides a composable set of transforms for this purpose.

```
import torchvision.transforms.v2 as T

# A preprocessing pipeline applied to every image before it enters the
    model
preprocess = T.Compose([
    T.ToImage(),                          # convert numpy array to torch
        tensor
    T.ToDtype(torch.float32, scale=True),# convert uint8 [0,255] to
        float [0,1]
    T.Resize((512, 512)),                 # resize to a fixed resolution
    T.Normalize(
        mean=[0.485, 0.456, 0.406],       # ImageNet channel means
        std= [0.229, 0.224, 0.225],       # ImageNet channel stds
    ),
])

# A data augmentation pipeline used only during training
augment = T.Compose([
    T.RandomHorizontalFlip(p=0.5),
    T.RandomVerticalFlip(p=0.5),          # aerial imagery has no
        canonical top
    T.ColorJitter(brightness=0.2, contrast=0.2),
])
```

Intuition

The `Normalize` step uses the channel means and standard deviations computed from the ImageNet dataset (a large collection of everyday photographs). We reuse these even for aerial imagery because most of our model's weights will have been pre-trained on ImageNet data (see below), and normalising with the same statistics the pre-training used keeps the input distribution consistent.

The `T.Compose` call chains transforms together so they run in sequence. The resulting object is callable – you apply it to an image like a function: `processed = preprocess(raw_image)`.

For object detection tasks, transforms need to be applied to both the image and the bounding boxes simultaneously (a random crop that removes part of the image must also update the box coordinates). The `v2` version of Torchvision transforms handles this automatically when you pass the target dictionary alongside the image.

Pre-trained models and transfer learning

Training a neural network from random initial weights on a dataset as small as NeonTreeEvaluation (around 6000 annotated trees) would not work well. There is not enough data for the model to learn low-level visual features like "edge", "texture", and "blob of similar colour" from scratch. Instead, we use *transfer learning*: start from a model that has already learned these features on a large dataset, then fine-tune it on our data.

Torchvision provides this out of the box:

```
from torchvision.models.detection import fasterrcnn_resnet50_fpn,
    FasterRCNN_ResNet50_FPN_Weights

# Load Faster R-CNN pre-trained on COCO (a large object detection
    dataset)
weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn(weights=weights)

# Replace the final classification head to detect only one class (tree
    crown)
# The rest of the network keeps its pre-trained weights
in_features = model.roi_heads.box_predictor.cls_score.in_features
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
    num_classes=2)
# num_classes=2 means: background (class 0) + tree crown (class 1)

model = model.to(device)
```

The pre-trained backbone (ResNet-50) has learned to recognise textures and shapes across millions of photographs. We discard only the final layer that maps features to COCO category labels, replace it with a fresh layer for our single class, and then train the whole thing on our aerial imagery. The backbone will update its weights slowly to adapt to the overhead perspective and tree-specific appearance while retaining the general visual understanding it was initialised with.

Inference: running the model on new data

During inference (applying the model to images it has not seen before) there is no need to track gradients, which saves memory and time. PyTorch provides a context manager for this:

```
model.eval()   # switch model to evaluation mode (disables dropout etc.)

with torch.no_grad():
    for images, _ in test_dataloader:
        images = [img.to(device) for img in images]
        predictions = model(images)
        # predictions is a list of dicts, one per image:
        # {"boxes": tensor of shape (N, 4),
        #  "labels": tensor of shape (N,),
        #  "scores": tensor of shape (N,)}

        for pred in predictions:
            keep = pred["scores"] > 0.5          # filter
                low-confidence detections
            boxes = pred["boxes"][keep].cpu().numpy()
            # boxes is now a numpy array of [x1, y1, x2, y2] coordinates
```

Note the `.cpu().numpy()` chain: we move the tensor back to CPU memory (from the GPU) and then convert to a NumPy array so we can work with it using standard tools.

The Full Pipeline in One View

To close, here is a sketch of how the pieces connect, from raw data on disk to a trained model producing predictions:

1. **Load** – `rasterio` reads a GeoTIFF tile and returns a NumPy array of shape `(channels, H, W)`.
2. **Package** – a custom `NeonDataset` wraps the file paths and knows how to load one image and its bounding-box annotations on demand.
3. **Preprocess** – Torchvision transforms convert the array to a float tensor, resize it, and normalise the pixel values.
4. **Batch** – a `DataLoader` collects 4 (or 8, or 16) processed images into a batch of shape `(B, C, H, W)`.
5. **Forward pass** – the batch goes into the model (Faster R-CNN or similar); PyTorch runs the computation on the GPU.
6. **Loss** – the model compares its predicted boxes against the ground-truth annotations and returns a scalar loss value.
7. **Backward pass** – `loss.backward()` computes gradients automatically.
8. **Update** – the optimizer nudges every parameter slightly in the direction that reduces the loss.
9. **Repeat** – steps 4–8 repeat for every batch, for multiple passes through the full dataset (called *epochs*).

The forestry team is most likely to be involved in steps 1–2 (loading and inspecting data, verifying that annotations look correct, splitting the dataset into train and test sets) and step 9 (deciding when the model's predictions are good enough for ecological conclusions). Steps 5–8 largely run on their own once the scaffold is in place.

Note

A recurring theme in this document is that you should always know the *shape* of every array or tensor you are working with. When code breaks, the first thing to do is print the shapes of the tensors involved. Most errors in ML code come down to a shape mismatch: two tensors that were expected to be compatible turned out to have different sizes along some dimension.